

EEE 306 PROJECT REPORT

A Comparative Study
of Machine Learning-
Based Methods for
Global
Horizontal Irradiance
Forecasting

Submitted by:

Group No.:05

Group members:

1. Fardin Ahmed
2. Most. Noor Afroz Rimu
3. Al Amin Hossain
4. Sabbir Ahammad
5. Biswas Rudra Jyoti Arka
6. Fariha Nusaibah

ID :1806056
ID: 1806054
ID: 1806055
ID: 1806057
ID: 1806058
ID:1806059

Table of Contents

Abstract	2
Overview	3
Dataset Acknowledgement.....	3
Data Visualization.....	4
Project Pipeline	6
1. Preprocessing.....	8
2. Splitting the Dataset	8
3. Training the Model	8
(i) LSTM	8
(ii)SNN.....	9
(iii)FNN.....	10
4. Fitting the Model	11
5. Error Evaluation	13
6. Ensembling the Model.....	13
7. Weighted Average	14
8. RMSE Score from other Classifiers(SVM)	14
Output Score Analysis	14
Discussion	15
References	15

Abstract

Grid operators are becoming more concerned about safety and service quality as **Photovoltaic (PV) Electricity Generation** in power distribution systems keeps spreading. The reliability of the power system is impacted by the intrinsic variability of PV power generation, which is mostly caused by weather circumstances. To develop efficient monitoring and control schemes for distribution grids, reliable forecasting of the solar resource at several time horizons that are related to regulation, scheduling, dispatching, and unit commitment, are necessary. PV power generation forecasting can result from forecasting **Global Horizontal Irradiance (GHI)**, which is the total amount of shortwave radiation is received from above by a surface horizontal to the ground. A comparative study of machine learning methods is given in this project.

As electricity is not easy to store; supply and demand must be always balanced by grid operators. Due to the intermittent nature of the solar resource, the deployment of **Photovoltaic (PV)** power generation makes the power grid balance more complex situations using conventional tools. This project, which uses a predictive management strategy, is a stepping-stone to more efficient real-time monitoring and optimization of grid operation.

Solar radiation is an important parameter for solar energy research but is not available for most of the sites due to the non-availability of solar radiation measuring equipment at the meteorological stations. Therefore, it is essential to predict solar radiation for a location using commonly measured meteorological variables. Solar radiation can be divided into three components at the ground: **global**, **diffuse**, and **direct**. The global component is the sum of the two others. While solar concentrator systems require knowledge of the direct solar component, the precise knowledge of the global horizontal solar irradiation is very important in the design, sizing, and production forecasting of thermal and photovoltaic solar systems.

The **GHI score** depends on several parameters such as **precipitation**, **atmospheric pressure**, **relative humidity**, **air temperature**, **wind direction**, and **wind speed**. But their proper relation is unknown. Our objective is to find a relation between GHI and these parameters. The **train.csv** file comes with an ID for each GHI recording event and 6 other parameters. We will now prepare an algorithm that can predict GHI with the minimum error possible. We will predict our model on the cross-validation set and measure the accuracy. The accuracy metric is the **RMSE score**. A lower RMSE score implies better and more accurate prediction.

We have used **Python** and some other python libraries (**NumPy**, **Pandas**, **TensorFlow**, **Matplotlib**, **SK-learn**) in this project. We used **LSTM (Long Short-Term Memory)**, **SNN (Spiking Neural Network)**, and **FNN (Feedforward Neural Network)** as our core training model and a stacked model of these three to reach the highest prediction-accuracy score. We used **RMSE Score** as our error evaluation parameters.

Overview

- **Visualization:** We observed dataset type, shape, data distribution type, correlation, and GHI value over time.
- **Preprocessing:** We used **standard scaling** for removing the mean and scaling the data to unit variance. We split the dataset **manually** for **LSTM** input because a **shuffled dataset** cannot work in LSTM input as LSTM works with time-dependent data. For the other models, we used **sklearn train_test_split**.
- **Model training:** We trained **three** different neural network models: **SNN**, **FNN**, and **LSTM**. All hyperparameters were tuned by **RANDOMIZEDSEARCHCV**.
- **Checking error:** We used the **RMSE** score as the error evaluation parameter.
- **Ensembling:** We used **stacking_ensemble** on the pre-trained model.
- **The weighted average on prediction:** We weighted the **stacked_model**, **SNN**, and **LSTM** outputs concerning their accuracy on the validation set.
- **Comparing with other baseline models:** We compared the RMSE score with a baseline SVM model and showed that our model gives around half error than that of the baseline model.

Dataset Acknowledgement

The primary source for the **Dataset-Bangladesh Solar Radiation Measurement Data**. The campaign that generated this data was commissioned by **The World Bank** with funding from the **Energy Sector Management Assistance Program (ESMAP)**. The World Bank's open data policy makes the data freely available. **Site Name:** BDFE2 (Feni)

Equipment: Helioscale omega station (Tier 1)

Host Institution: Char Darbesh Adarsha Gram Government Primary School

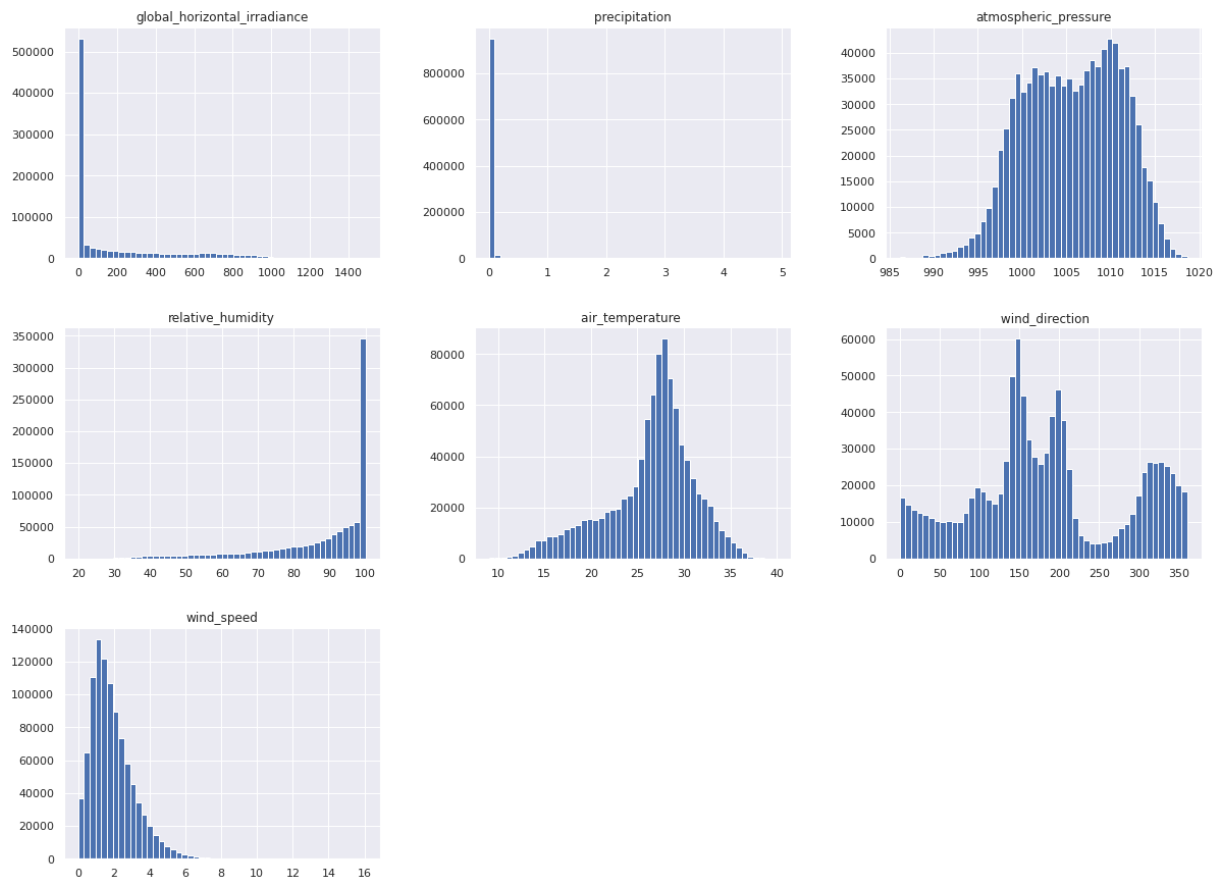
Elevation (m): 5

Latitude (positive North decimal degrees): 22.80029

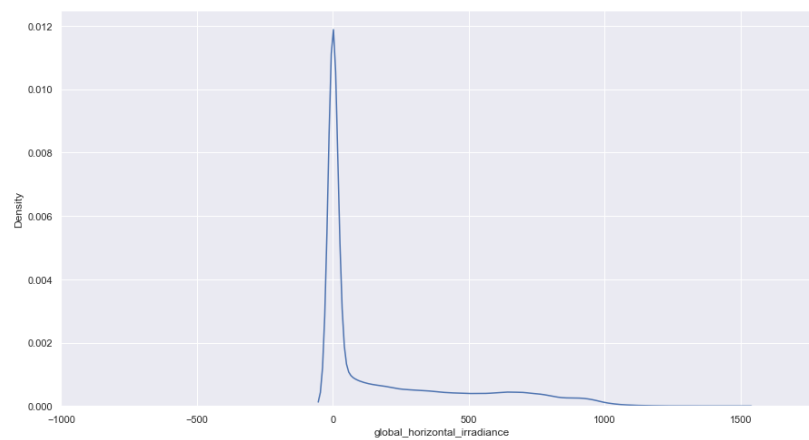
Longitude (positive East decimal degrees): 91.35819

Data Visualization

We can see Normal **distribution** in **air temperature** and **pressure**. **Poisson distribution** in **wind speed**. We will scale all features later.

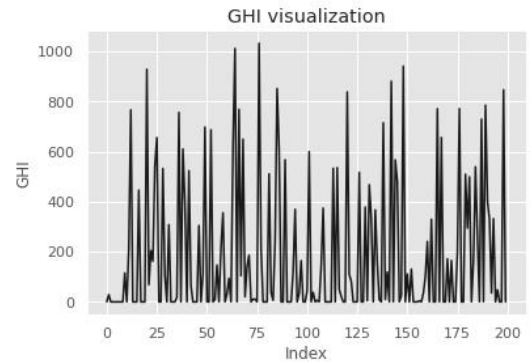
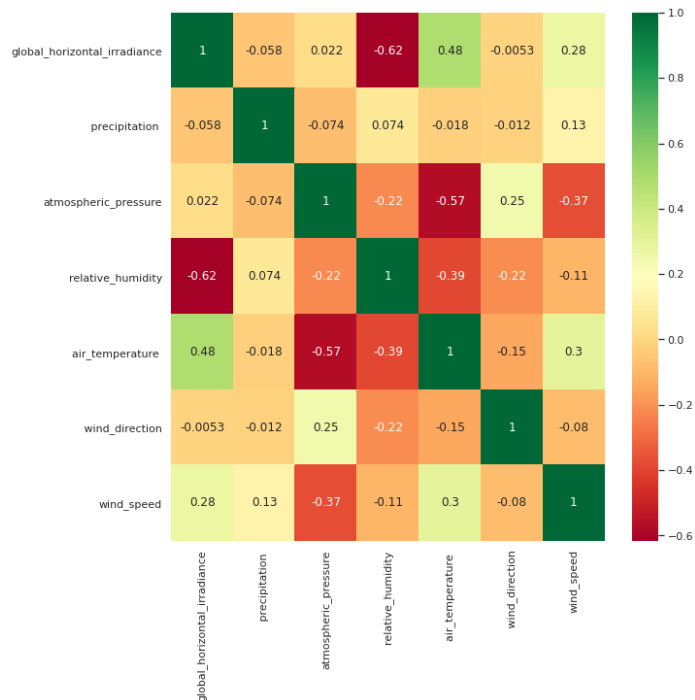


We can see from the next graph most of the **GHI value** is distributed in zero.



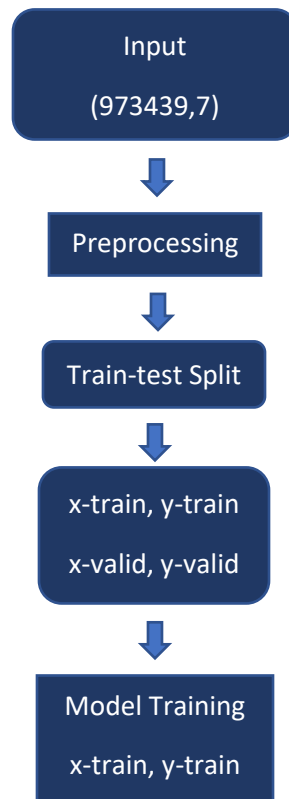
Correlation between features for more insight:

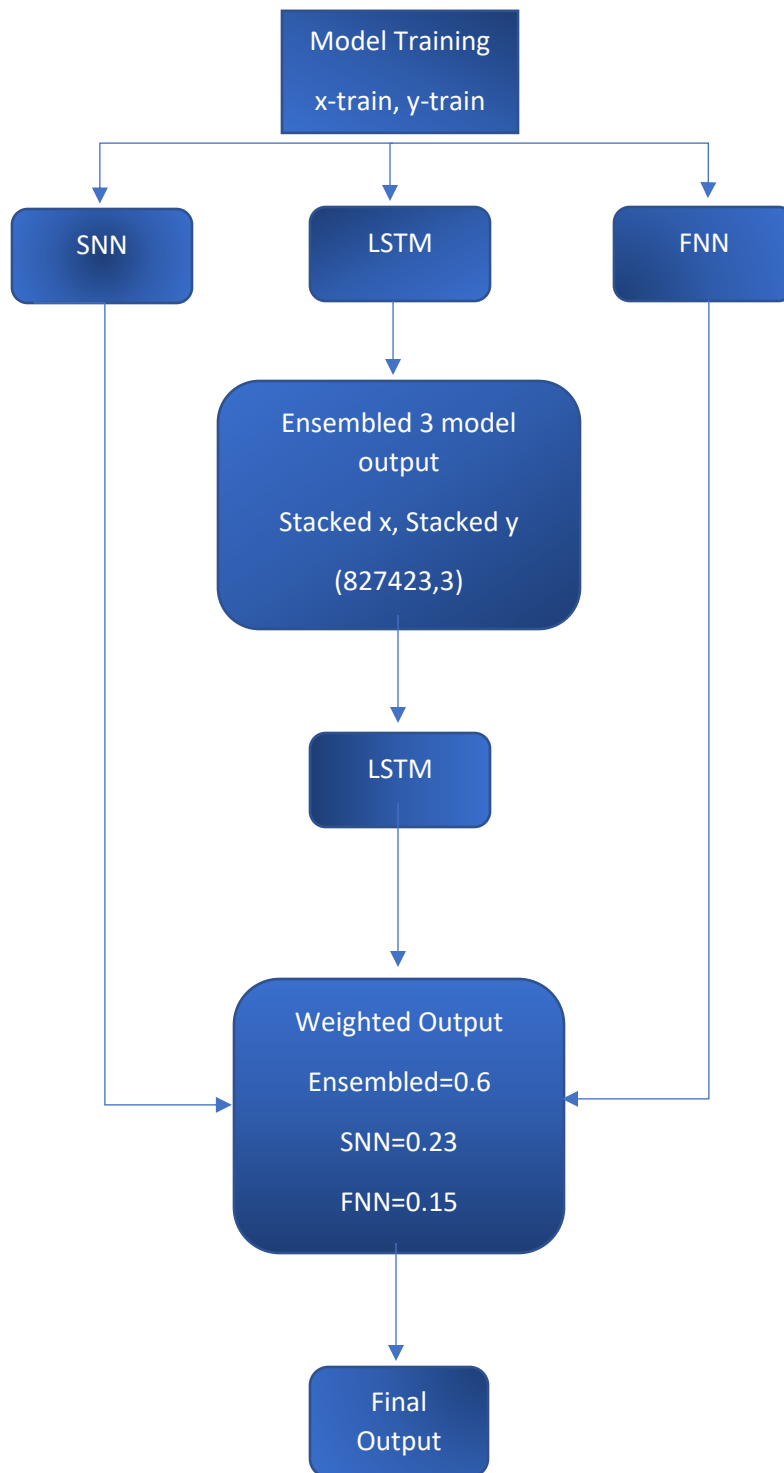
We can see the wind direction and precipitation is less correlated with GHI. But we cannot drop those columns because they have an impact on solar prediction. Moreover, dropping columns hampers model accuracy.



As GHI is time series data, we can use the LSTM model.

Project Pipeline:





1. Preprocessing

We used **1. Log transformation 2. Square root transformation 3. Boxcox transformation 4. Standard scaling** on **train_data**. Among them, we find **standard scaling** does better than all of them. Standard scaling is a scaling technique where the values are centered around the mean with a unit standard deviation. This means that the mean of the attribute becomes zero and the resultant distribution has a unit standard deviation.

2. Splitting the dataset

We put **85%** of data as **train_data** and **train_label** and the other **15%** of data as **x-valid** and **y-valid**. The final sizes are train_data (827423, 6), train_label (827423,), x_valid (146016, 6) and y_valid (146016,).

3. Training the model

We used a **sequential model** in this workflow. A Sequential model is appropriate for a plain stack of layers where each layer has exactly one input tensor and one output tensor. We also had to take some measures to avoid the **overfitting** problem. Overfitting is an error that occurs in data modeling as a result of a particular function aligning too closely to a minimal set of data points. To avoid this problem, we used **regularizers** and **dropout**. Regularization refers to techniques that are used to calibrate machine learning models in order to minimize the adjusted loss function and prevent overfitting or underfitting. Dropout is a regularization method that approximates training a large number of neural networks with different architectures in parallel. We used **Lasso** and **Ridge** regularizers.

LSTM: Long short-term memory (LSTM) is an artificial neural network used in the fields of artificial intelligence and deep learning. Unlike standard feedforward neural networks, LSTM has feedback connections. LSTM is used to analyze time-dependent data.

Lasso regularizers (L1) = 0.001 Ridge regularizers (L2) = 0.01 Layers = 5, Activation Function = SELU, Initializer = Lecun_normal, Dropout= 0.2 & 0.1

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
lstm (LSTM)	(None, 1, 500)	1014000

dropout (Dropout)	(None, 1, 500)	0

batch_normalization (BatchNo	(None, 1, 500)	2000

lstm_1 (LSTM)	(None, 1, 300)	961200

batch_normalization_1 (Batch	(None, 1, 300)	1200

dropout_1 (Dropout)	(None, 1, 300)	0
lstm_2 (LSTM)	(None, 1, 150)	270600
batch_normalization_2 (Batch Normalization)	(None, 1, 150)	600
dropout_2 (Dropout)	(None, 1, 150)	0
lstm_3 (LSTM)	(None, 1, 150)	180600
batch_normalization_3 (Batch Normalization)	(None, 1, 150)	600
lstm_4 (LSTM)	(None, 100)	100400
dense (Dense)	(None, 1)	101
=====		
Total params: 2,531,301		
Trainable params: 2,529,101		
Non-trainable params: 2,200		

SNN: Spiking neural networks (SNNs) are artificial neural networks that more closely mimic natural neural networks. In addition to neuronal and synaptic states, SNNs incorporate the concept of time into their operating model.

Lasso regularizers (L1) = 0.001 Ridge regularizers (L2) = 0.01 Layers = 8, Activation Function = SELU, Initializer = Lecun_normal, Dropout= 0.2 & 0.1

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
dense_1 (Dense)	(None, 600)	4200
dense_2 (Dense)	(None, 600)	360600
dense_3 (Dense)	(None, 300)	180300
dense_4 (Dense)	(None, 300)	90300
dense_5 (Dense)	(None, 150)	45150
batch_normalization_4 (Batch Normalization)	(None, 150)	600
dense_6 (Dense)	(None, 150)	22650
batch_normalization_5 (Batch Normalization)	(None, 150)	600
dense_7 (Dense)	(None, 150)	22650
batch_normalization_6 (Batch Normalization)	(None, 150)	600

dense_8 (Dense)	(None, 100)	15100
dense_9 (Dense)	(None, 1)	101
=====		
Total params: 742,851		
Trainable params: 741,951		
Non-trainable params: 900		

FNN: A feedforward neural network (FNN) is an artificial neural network wherein connections between the nodes do not form a cycle. As such, it is different from its descendant: recurrent neural networks.

Lasso regularizers (L1) = 0.001 Ridge regularizers (L2) = 0.01 Layers = 5, Activation Function = ELU, Initializer = HE_normal, Dropout= 0.1

Model: "sequential_2"

Layer (type)	Output Shape	Param #
=====		
dense_10 (Dense)	(None, 500)	3500
batch_normalization_7 (Batch Normalization)	(None, 500)	2000
dense_11 (Dense)	(None, 500)	250500
batch_normalization_8 (Batch Normalization)	(None, 500)	2000
dense_12 (Dense)	(None, 300)	150300
batch_normalization_9 (Batch Normalization)	(None, 300)	1200
dense_13 (Dense)	(None, 300)	90300
batch_normalization_10 (Batch Normalization)	(None, 300)	1200
dropout_3 (Dropout)	(None, 300)	0
dense_14 (Dense)	(None, 150)	45150
batch_normalization_11 (Batch Normalization)	(None, 150)	600
dense_15 (Dense)	(None, 150)	22650
batch_normalization_12 (Batch Normalization)	(None, 150)	600
dense_16 (Dense)	(None, 150)	22650
batch_normalization_13 (Batch Normalization)	(None, 150)	600
dense_17 (Dense)	(None, 100)	15100
dense_18 (Dense)	(None, 1)	101

```
=====
Total params: 608,451
Trainable params: 604,351
Non-trainable params: 4,100
```

4. Fitting the Model

The dataset was fit into the model through 3 steps:

Step 1: Epoch=130, Batch size=250

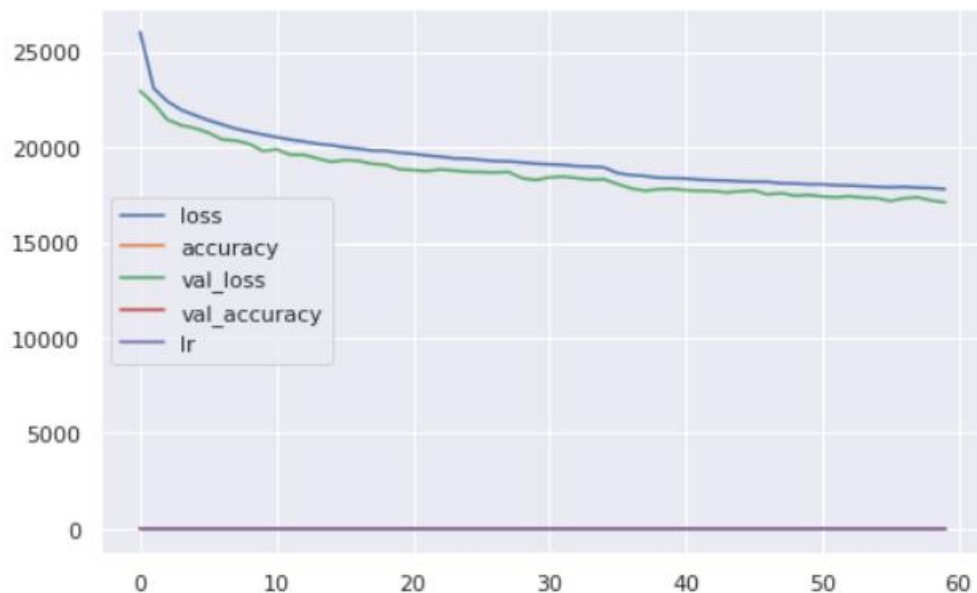
Step 2: Epoch=60, Batch size=512

Step 3: Epoch=40, Batch size=1024

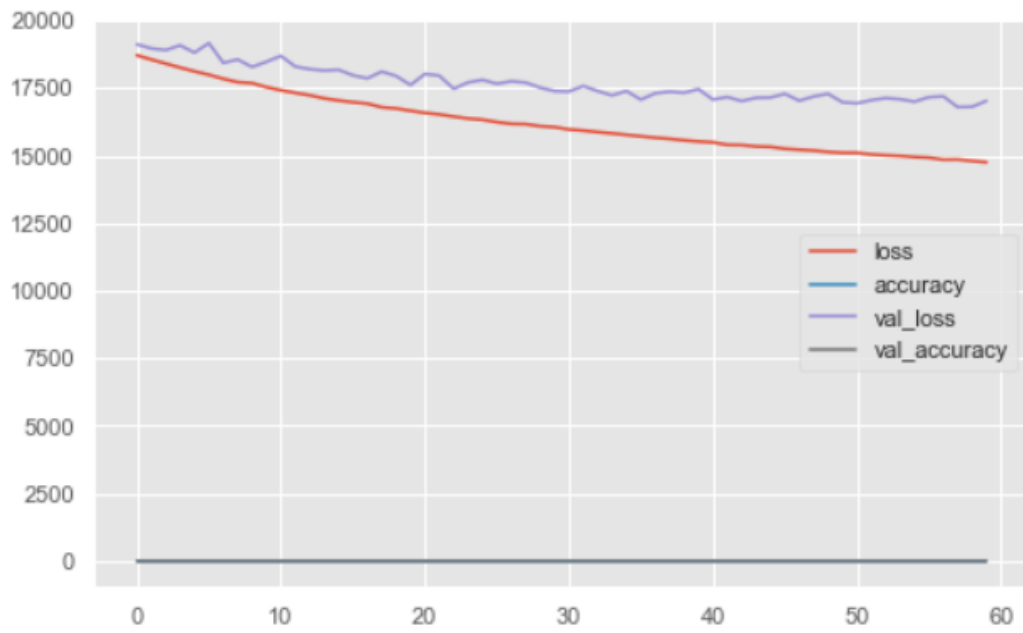
We had to take some extra measures to avoid the **Stucked At Plateau** Problem. This is a phenomenon where the loss value stops decreasing during the process of learning and decelerates the performance of the optimizer algorithm. To avoid this problem, we defined a callback called **reduce_lr** with parameters such as **patience= 5,4,3** and **factor= 0.7,0.5,0.3**

After fitting, the acquired learning curves for the 3 models are given below.

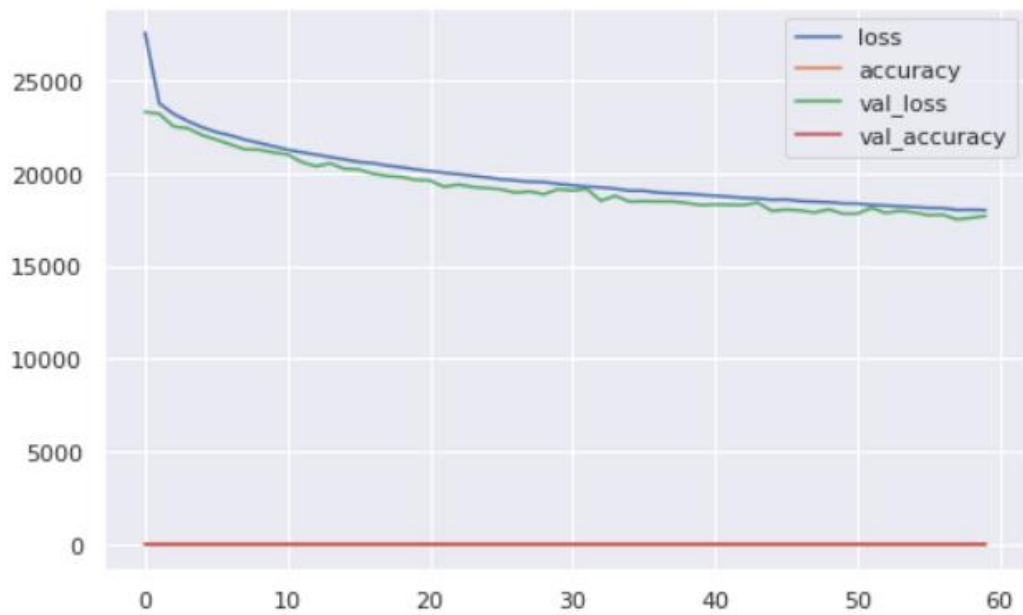
LSTM :



SNN:



FNN:



5. Error Evaluation

We have used **RMSE Score** to evaluate the error percentage of our project. **Root Mean Square Error (RMSE)** is the standard deviation of the **residuals** (prediction errors). Residuals are a measure of how far from the regression line data points are; RMSE is a measure of how spread-out these residuals are.

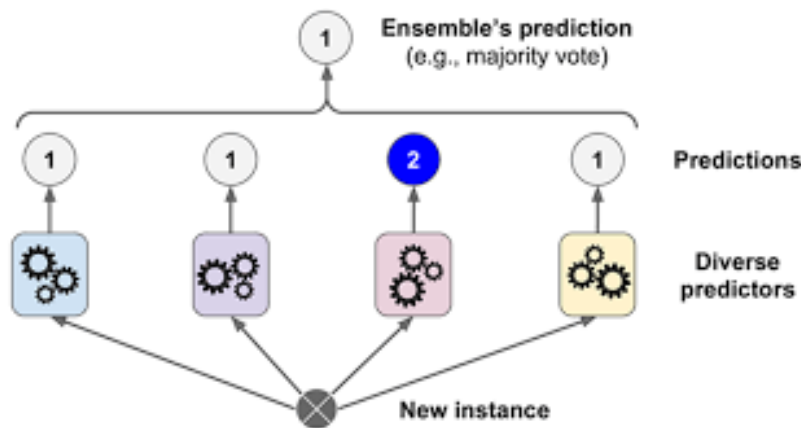
The **Train RMSE** and the **Validation RMSE** of the 3 models are shown in the table below.

Model Name	Train RMSE	Validation RMSE
LSTM	136.38760362750918	139.3469699947232
SNN	103.24288363861389	109.45354372986293
FNN	111.61869671722715	114.69376671435116

From the table, we can see that **SNN** and **FNN** have **better** RMSE scores than **LSTM**.

6. Ensembling the model

Now we are going to load the three models that we have trained, and ensemble them to make a new model which we will use for better prediction.



Here in this part, we have taken the output from the 3 previously trained models and put them together in a new stacked model which we call the **Ensembled Model**. The output of this model has a dimension of (146016, 3).

Next, we passed the output of the Ensembled Model through LSTM again. The **validation RMSE** of this model is **107.283104** which is already better than the previous 3 models working alone (**LSTM**: 139.346969 **SNN**: 109.453543 **FNN**: 114.693766) .

7. Weighted Average

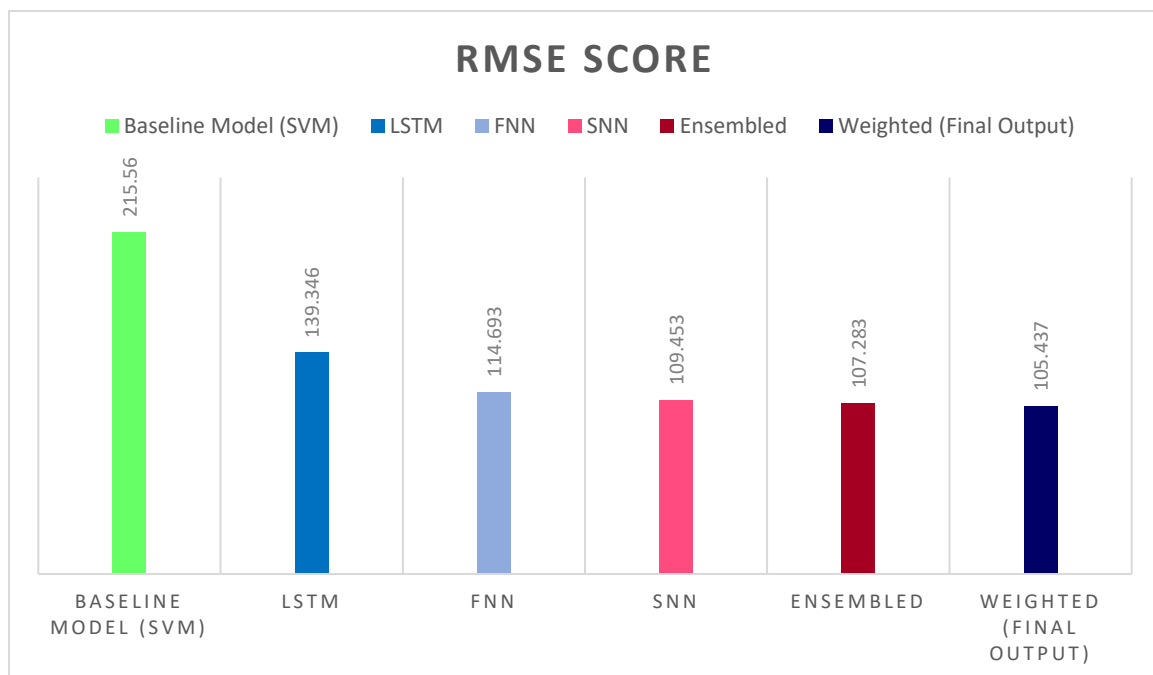
Next, we took a weighted average of the **Ensembled** output, the **SNN** output, and the **FNN** output. We multiplied the Ensembled output by 0.6 as it had the highest validation RMSE, and for SNN and FNN, 0.25 and 0.15 respectively.

After this process, we had the final output of our project which has the final RMSE Score of **105.43752582424912** which is a better score than all the previous steps.

8. RMSE Score from other Classifiers (SVM)

We trained the **Support Vector Machine (SVM)** which is a very well-known classifier and its RMSE score was **215.5582023407721**. Here we can see, that if we use SVM regression, without tuning any hyperparameter, **the error is nearly double of that of our model output**. So, we can say that our model works far better than those classifiers.

Output Score Analysis:



Field of Application

Global horizontal irradiance (GHI) is the most useful metric for predicting solar panel output. It accounts for **71.6% of PV performance variations**. Accurate **variable renewable energy (VRE)** forecasting for both short-term and long-term conditions benefits the entire solar industry for transmission system planning and stability for operators. Weather forecasting data is called the '**Most Critical Scheduling Input**' for a solar power generation forecast.

Forecasting using GHI has a greater impact on solar cell operating cost reduction. VRE forecasting is most used in centralized systems, as a best practice. This type of forecasting covers all generators in a balancing area across an entire grid system. It is generally preferred to decentralized forecasts because it leads to consistency, reduced uncertainty, and a lower financial burden for VRE plants.

In Bangladesh, we have **8 solar power plants** already running and 31 more in implementation and planning state. As we don't have plenty of lands to implement solar power plants, these ongoing solar projects' sites could be selected where VRE forecasting provides maximum GHI. This can boost our solar plants' efficiency to the maximum level. Our model can train and predict data for every district in Bangladesh which can help a lot in this purpose.

Discussion

Our project has been conducted through machine learning to forecast Global Horizontal Irradiance. In our project, we have used standard scaling as preprocessing. 85% of our data were selected as group data and others worked as test data. We've used LSTM, SNN, and FNN to make our prediction. While conducting our project, we have faced some problems like an error that occurs in data modeling as a result of a particular function aligning too closely to a minimal set of data points which is called overfitting. To solve this problem, we used regularizers and dropout. From our result, it was evident that SNN and FNN have better RMSE scores than LSTM. We also took the output from the 3 previously trained models and put them together in a new stacked model which we call the Ensembled Model. After passing it through another LSTM it was seen that the error had been minimized more. After taking a weighted average of SNN, FNN, and ensembled model's output we got our final minimized result. It is clearly seen that the RMSE score of the Support Vector Machine (SVM) was far higher than the output of our model. Hence, we can say that the prediction of our model is more satisfying than the normal Support Vector Machine (SVM).

References:

1. <https://ieeexplore.ieee.org/document/9244163>
2. <https://aip.scitation.org/doi/pdf/10.1063/1.5084997>